

MinimalAuth

Gavriluță Rareș-Andrei

¹ Faculty of Computer Science, Alexandru Ioan Cuza University, Iași
rares.gavriluta@info.uaic.ro

Abstract. MinimalAuth consists of a minimal use case for the OAuth protocol. It allows a third-party client to obtain limited access to the resource files of a proprietary server by orchestrating a client authorization interaction, thus avoiding the disclosure of the password that the user normally uses.

Keywords: OAuth, token, security, resource files, authorization, client

1 Introduction

In the traditional client-server authentication model, the client requests a restricted resource (protected resource) located on the server by authenticating with the server using the user's login credentials. Therefore, in order to provide third-party applications with access to protected resources, the resource owner must share their authentication data with the third-party application in question. This causes a number of problems and limitations:

- Third-party applications (clients) are required to store the user's authentication data for later use, usually a password in plain text.
- Servers must accept password authentication, despite the inherent security weaknesses of passwords.
- Third-party applications gain overly broad access to protected resources, leaving owners with no ability to restrict the duration or access to a limited subset of resources.
- Resource owners cannot revoke access to an individual third party without revoking access to all third parties and must do so by changing the third party's password.
- Compromising any third-party application implicitly compromises the user's password and all data protected by that password.

The MinimalAuth protocol addresses these issues by introducing a level of authorization and separating the role of the client from that of the resource owner. In MinimalAuth, the client requests access to protected data hosted by the resource server and is granted a set of credentials, different from those typically used by the owner user.

Instead of using the owner's credentials to access protected resources, the client obtains an access token—a string that denotes a specific purpose, its lifetime, and other possible attributes. Access tokens are issued to third-party clients by an authorization server with the approval of the resource owner. The client then uses this access token to access protected resources hosted by the resource server.

For example, a user (resource owner) can authorize a printing service (client) to access protected photos hosted by a photo sharing service (resource server) without sharing their username and password with the printing service. Instead, the user authenticates to a verified and trusted server (authorization server) for the printing service, which then issues a set of credentials (access token) to the printing service.

2 Technologies used

2.1 TCP servers

For both the authorization server and the resource server, we implemented a TCP server instead of a UDP server for the following reasons:

- TCP is connection-oriented in the sense that, before two processes can transmit data, they must "get to know" each other (a preliminary exchange of data that establishes the parameters for the upcoming transfer). In our case, the entire authorization protocol requires a direct connection between clients and servers.
- TCP guarantees that data is not damaged, lost, duplicated, or delivered incorrectly between the processes involved in the transfer. Our clients make requests to receive bulk files in return, and TCP is the recommended protocol for this type of transfer.
- The TCP/IP model provides a flow control mechanism, which basically means that TCP will ensure that a sender does not overwhelm a receiver by sending packets faster than it can consume. Since our protocol frequently transfers (large) files, we do not want the client's receive buffer to be overloaded by the transmitting server.

2.2 Concurrent TCP server architecture – thread per client

For the authorization server that clients will connect to in order to request an access token, we chose a thread-per-client architecture. This type of architecture is suitable for two main reasons. First, considering that our server will be frequently requested, the alternative architecture (process per client) that uses `fork()` is not recommended due to performance (the `fork()` method is relatively expensive to execute). In addition, in terms of execution time, threaded implementation is generally faster than other similar architectures, including the optimized "preforked – process per client" variant.

Table 1. Comparison between the execution times of some TCP clients

TCP client	Execution time (sec.)
Usual pattern (stop-and-wait)	...
Using select and blocking I/O operations	12.3
Using select and unblocking I/O operations	6.9
Using fork()	8.7
Using threads	8.5

2.3 Prethreaded concurrent TCP server with mutex locking

For the implementation of the server that hosts resources (protected files), we have introduced, as an additional optimization, a concurrent TCP server that serves clients through a prethreading mechanism and also uses the mutex lock protection of `accept()`. Compared to the authorization server, it creates a number of threads chosen by us as soon as it is started, before clients connect. The optimization comes from the fact that the mechanism by which we pre-create threads that can call `accept()` is faster and simpler than the one in which the main thread calls `accept` and also redirects the descriptor to the other threads/child processes. The locking mechanism used (the mutex lock) ensures that there cannot be two threads calling `accept()` simultaneously, a scenario we do not want.

Given that we can specify the number of threads to be created as a parameter, this allows us to prioritize performance over latency, if we so desire.

3 Protocol architecture

3.1 Protocol diagram

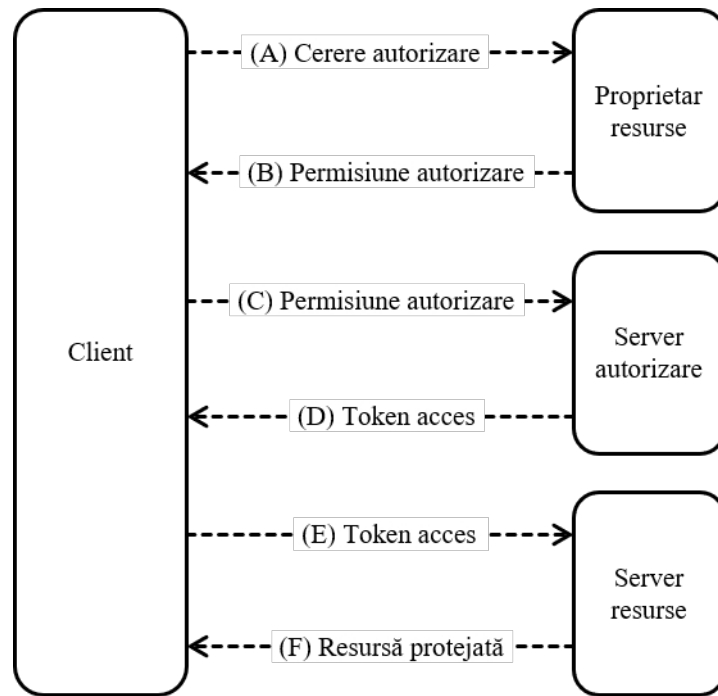


Fig. 1. Abstract application diagram

Figure 1 illustrates the abstract application diagram and the interactions between the main actors as follows:

- (A) The client makes an authorization request to the resource owner with the intention of accessing the resources protected by it. In our case, as an additional security method, the request is not made directly, but indirectly, through the authorization server (a process that will be described below).
- (B) The client receives authorization permission, which can be expressed in several ways. In our case, the permission is a numeric authorization code that is used further on.
- (C) The client requests an access token from the authorization server, presenting the authorization code received earlier.
- (D) If the code presented by the client matches the one issued, the authorization server generates an access token, a number of uses for it, an optional refresh token, validates the generated tokens, and transmits them to the client.
- (E) The client connects to the resource server, presents the access token received, and requests a protected file.

- (F) The resource server verifies that the token is valid and has not expired, and if so, serves the client with the requested resource.

3.2 Concepts involved

Authorization code. The authorization code is the means by which the resource owner grants the client permission to request an access token. Instead of requesting permission directly from the owner, the client redirects the owner to the authorization server, which generates a one-time code, and the latter (authorization server) redirects the owner back to the client to transmit this code. In this step, the owner must authenticate or already be authenticated before being granted the code to transmit to the client. Since the only authentication takes place between the owner user and the authorization server, the password is not disclosed to any foreign client and remains protected. In this case, we will assume that the user is already authenticated on the authorization server. This mechanism brings several important security benefits, including:

- the authorization server's guarantee that the client requesting the token is the one entrusted by the owner
- The ability of the authorizer to transmit the access token directly to the client, without other intermediaries (including the owner) who could compromise it.

Access token. Access tokens are a form of credential granted to allow access to protected resources. They are usually generated in the form of strings, opaque to the client, and denoting several attributes such as purpose, number of uses, etc. (similar to a CNP). These attributes can be passed through an abstraction layer so that they are only understood by the resource server.

Refresh token. Refresh tokens are assigned along with access tokens and are intended to be used when the access token expires or becomes invalid. Their usefulness stems from the fact that once authorized, the client no longer needs to go through the entire protocol to obtain a new access token from the server, but can use it instead of the authorization code.

3.3 Granting the authorization code

To request an access token, the client obtains authorization permission from the resource owner. This permission is expressed through an authorization code, which the client uses to request tokens.

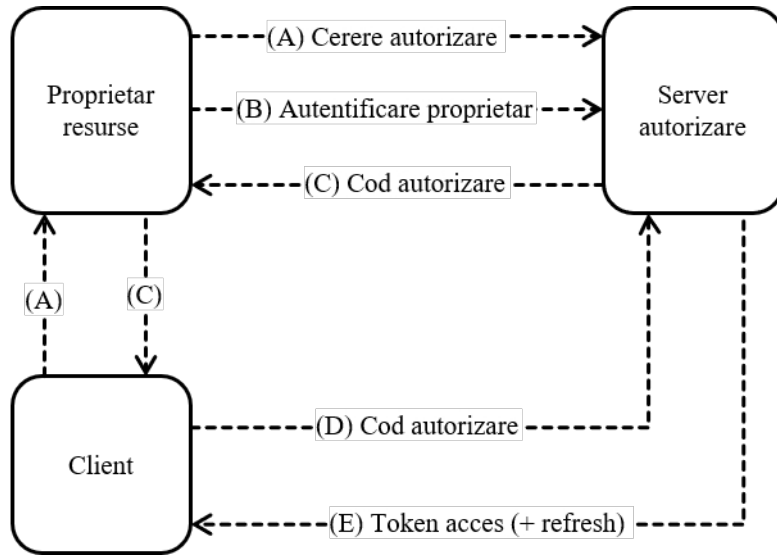


Fig. 2. Authorization code granting diagram

The diagram shown in Figure 2 includes the following steps:

- (A) The client requests permission to request a token by redirecting the user to the authorization server to obtain an authorization code
- (B) The authorization server ensures that the user is authenticated and determines whether the owner actually wants to authorize the client
- (C) If so, the server generates an authorization code and redirects the user back to the client with this numeric code
- (D) The client takes the code and sends it to the authorization server to obtain an access token.
- (E) The authorization server verifies the code received and, if it matches the one issued earlier, serves the client with the requested token and, optionally, with a refresh token.

4 Implementation details

4.1 Protocol-specific code

The client redirects the user to the auth_server. It transmits the pid to the auth_server as a method of identification

```

/* start */
printf("[client] Authorizing client %d\n", pid); printf
("Redirecting you to auth_server...\nRead
the code received from [auth_server]\n");
fflush (stdout);

```

```

    /* connecting to the server */
    if (connect (sd, (struct sockaddr *)
&server,sizeof (struct sockaddr)) == -1)
    {
        perror ("[client] Error in connect() auth_ser-
ver");
        return errno;
    }

    if (write (sd, &pid, sizeof(int)) <= 0)
    {
        perror ("[client] Error writing() pid to
auth_server");
        return errno;
    }

```

Auth_server requests confirmation from the resource owner. If so, it generates an authorization code and sends it to the user. Otherwise (the code received later does not match or the user refuses authorization), auth_server will reject the client.

```

int grant_access(void *arg)
{
    int pid; int auth_code, cl_code=0;
    char buf[100];

    struct thData tdL;
    tdL= *((struct thData*)arg);

    if (read (tdL.cl, &pid, sizeof(int)) < 0)
    {
        perror ("[client] Error at read() from client");
        return errno;
    }

    printf("[thread %d] Client with pid: %d requests access
to resources\n Confirm? (y/n)\n", tdL.idThread, pid);
    gets(buf);
    if(strcmp(buf, "y")==0)
    {
        auth_code=rand()/1000000;
        printf("[thread %d] Enter this code in client(%d) :
%d\n", tdL.idThread, pid, auth_code);
        read(tdL.cl, &cl_code, sizeof(int));
        if(auth_code==cl_code)
            return pid;
    }
}

```

8

```
        else
        {
            printf("Client denied. %d Code does not match %d\n",
cl_code);
            return 0;
        }

    }
    else
    {
        printf("Client denied.\n");
        return 0;
    }
}
```

The authorization server validates the client. Tokens are generated, validated, and sent to the client

```
if((pid = grant_access((struct thData*)arg))!=0)
{
    char * token=generate_token();
    char * refresh=generate_token();
    int token_uses=generate_attempts();

    validate_token((struct thData*)arg, token, refresh,
nr_utilizari_token, pid);
    printf ("[thread %d] Token generated: %s - %d attempts
- refresh: %s\n",tdL.idThread, token, token_uses, refresh);
    respond((struct thData*)arg, token, refresh);
}
```

The resource server serves clients that present a valid token. The token presented with each request is checked, and if it is valid, the number of uses is decremented and the requested file is transmitted. Otherwise, the client will be notified that the token has expired.

```
/* receives and resends a message to a client */
int serveClient(int client, FILE* tokens, int i)
{
    char buffer[100];    /* message */
    int bytes;          /* number of bytes read/written */
    char token_read[13];
    char fileName[30];
```

```

    char msgrasp[100]=" ";          //response message for
client
    char line[40];

    bytes = read (client, buffer, sizeof (buffer));
    if (bytes < 0)
    {
        perror ("[thread] Error at read() from client.\n");
        return 0;
    }

    strncpy(token_read, buffer, 12);
    strcpy(fileName, buffer+13);
    printf ("[thread %d] Received: %s - File req: %s\n", i,
token_read, fileName);
    int found=0;
    while(fscanf(tokens, "%s", line) &&& !feof(tokens))
    {
        if(strstr(line, token_read)!=0 &&&
atoi(line+13)>0)
        {
            printf("[thread %d] %s valid - Attempts
remaining: %d\n",i, token_read, atoi(line+13)-1);
            found=1;
            decrease_attempts(tokens, line);
            sendFile(client, fileName, i);
            break;
        }
    }
    if(found==0)
    {
        write(client, "Token expired!", 14);
        rewind(tokens);
        return 0;
    }
    return 1;
}

```

The client makes successive requests to the resource server. After receiving the access token, the client will request from res_server, successively and at intervals (for demonstration purposes), one randomly selected file at a time, until the token expires.

10

```
do
{
    const char * fileName = pick_file();
    sprintf(sendBuf, "%s %s", token,
    fileName); int len =strlen(sendBuf);
    sendBuf[len]='\0';
    printf("=====\n");
    printf("[client] Connecting to resource server: %s\n",
sendBuf);
    if (write (sd_res, sendBuf, strlen(sendBuf)+1) &lt;= 0)
    {
        perror ("[client] Error writing() to resource server\n");
        return errno;
    }

    if(read(sd_res, buff, 15) &lt; 0)
    {
        perror ("[client] Error reading() from
res_server");
        return errno;
    }
    else
    {
        printf("[client] %s\n",buff);
        if(strstr(buff, "Token expired!")==0)
            receiveFile(fileName, sd_res);
    }
    sleep(5);
}
while(strstr(buff, "Token expired!")==0);
```

4.2 Usage scenarios

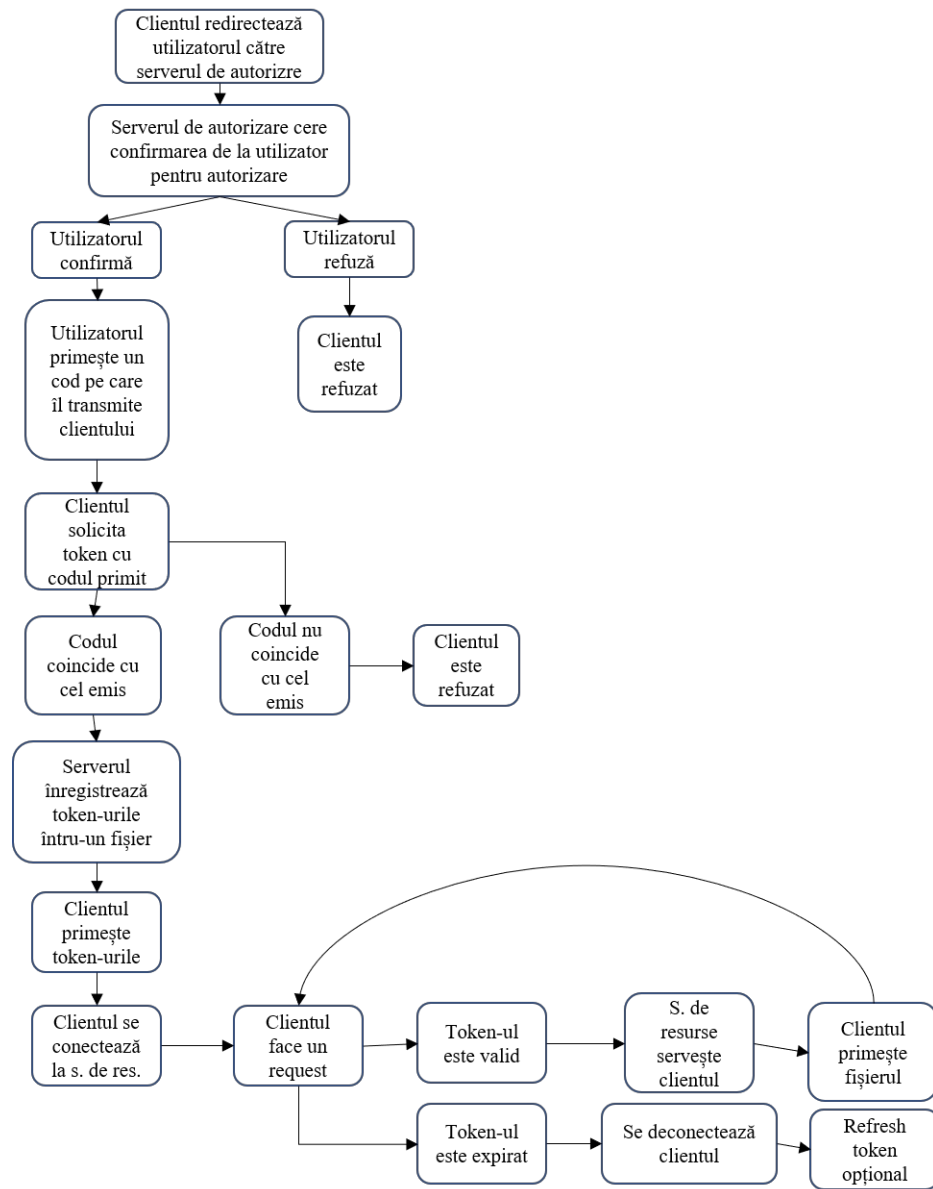


Fig. 3. Usage scenarios

5 Conclusions

The protocol presented above solves a security problem present in classic authentication through third-party applications and can be successfully implemented in many cases (web applications, desktop applications, banking applications, smartphones, smart devices in general, etc.).

However, it can be improved to enhance security and performance. One of the solutions we have partially implemented is refresh tokens. These mean that once authorized, the client no longer has to go through the entire protocol to obtain a new access token from the server, but can use it instead of the auto-refresh code. This mechanism can optimize, for example, a scenario in which we want to grant the client a token with a certain number of attempts, but only within a fixed time interval.

Another possible improvement can be made at the authorization code level. The Proof Key for Code Exchange (PKCE) extension describes a technique for clients to minimize the risk of authorization code interception. The technique requires the client to first create a secret code and then reuse that secret code when exchanging the authorization code for an access token. This way, if the code is intercepted, it will be useless because the token request is based on the original secret code.

6 References

1. <https://datatracker.ietf.org/doc/html/rfc6749>
2. <https://en.wikipedia.org/wiki/OAuth>
3. <https://developer.okta.com/blog/2017/06/21/what-the-heck-is-oauth>
4. <https://www.udemy.com/course/oauth-2-simplified/>